PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 2

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**Quick Sort (Movie Recommendation)**	**Batch:**	A1

1. Problem Statement

Movie Recommendation System Optimization: A popular OTT platform, StreamFlix, offers personalized recommendations by sorting movies based on user preferences, such as IMDb rating, release year, or watch time popularity. During peak hours, sorting large datasets slows down response latency. As a backend engineer, you must implement **Quick Sort** to efficiently sort movies based on various user-selected parameters while maintaining high throughput.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory

Quick Sort is an efficient, in-place, divide-and-conquer sorting algorithm. Unlike Merge Sort, which requires O(n) auxiliary memory for combining sorted lists, Quick Sort partitions the dataset directly in-place around a chosen **pivot element**.

Algorithmic Phases:

- **Divide (Pivot Selection & Partitioning):** A pivot element is selected. The array is partitioned such that all elements smaller than the pivot reside on its left, and all elements greater reside on its right. The pivot settles into its final sorted position.
- **Conquer:** The left and right sub-partitions are recursively sorted using Quick Sort.
- **Combine:** No separate combine phase is required because the array is sorted in-place.

Pivot Selection & Optimizations:

- **Last/First Element:** Simple but can lead to $O(n^2)$ worst case if the array is already sorted or reverse-sorted.

- **Median-of-Three:** Pivot chosen as the median of the first, middle, and last elements, providing balanced partitions in practice.
- **Randomized Pivot:** Choosing a random pivot prevents adversarial worst-case inputs.

Case	Time Complexity	Partition Balance
Best Case	O(n log n)	Pivot consistently partitions the array into two equal halves.
Average Case	O(n log n)	Random permutations yield balanced partitions on average.
Worst Case	$O(n^2)$	Occurs when pivot is always the smallest or largest element (highly unbalanced).
Space Complexity	O(log n)	Auxiliary space required by the recursion stack.

5. Quick Sort Algorithm & Pseudo Code

```
procedure QUICK_SORT(arr, low, high, choice):
    if low < high then
        pivotIndex ← PARTITION(arr, low, high, choice)
        QUICK_SORT(arr, low, pivotIndex - 1, choice)
        QUICK_SORT(arr, pivotIndex + 1, high, choice)
    end if
end procedure

procedure PARTITION(arr, low, high, choice):
    pivot ← arr[high]
    i ← low - 1
    for j ← low to high - 1 do
        if shouldComeBefore(arr[j], pivot, choice) then
            i ← i + 1
            SWAP(arr[i], arr[j])
        end if
    end for
    SWAP(arr[i + 1], arr[high])
    return i + 1
end procedure
```

6. Process Flow Diagram



7. Step-by-Step Example Walkthrough

Array of IMDb Ratings: [8.2, 6.5, 9.0, 7.3, 8.7]
• **Step 1 (Pivot Selection):** Choose pivot = 8.7 (last element).
• **Step 2 (Partition):** Elements ≤ 8.7 → [8.2, 6.5, 7.3], Elements > 8.7 → [9.0]. Array becomes: [8.2, 6.5, 7.3, 8.7, 9.0].
• **Step 3 (Recursive Sort Left):** Subarray [8.2, 6.5, 7.3] with pivot = 7.3 → [6.5, 7.3, 8.2].
• **Final Result: [6.5, 7.3, 8.2, 8.7, 9.0]**.

8. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 2: Movie Recommendation Data Sorting using Quick Sort
 */

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

struct Movie {
    int movieId;
    string title;
    string genre;
    int releaseYear;
    double imdbRating;
    long long watchTime;
};

// Swap two movies
void swapMovies(Movie &a, Movie &b) {
    Movie temp = a;
    a = b;
    b = temp;
}

// Get value according to user's choice
bool shouldComeBefore(const Movie &a, const Movie &b, int choice) {

    if (choice == 1) {
        // IMDb Rating
        return a.imdbRating < b.imdbRating;
    }
    else if (choice == 2) {
        // Release Year
        return a.releaseYear < b.releaseYear;
    }
    else {
        // Watch Time
        return a.watchTime < b.watchTime;
    }
}

// Partition function
int partition(Movie movies[], int low, int high, int choice) {

    Movie pivot = movies[high];

    int i = low - 1;

    for (int j = low; j < high; j++) {

        if (shouldComeBefore(movies[j], pivot, choice)) {
            i++;
            swapMovies(movies[i], movies[j]);
        }
    }

    swapMovies(movies[i + 1], movies[high]);

    return i + 1;
}
```

```
// Quick Sort
void quickSort(Movie movies[], int low, int high, int choice) {

    if (low < high) {

        int pivotIndex = partition(
            movies,
            low,
            high,
            choice
        );

        quickSort(
            movies,
            low,
            pivotIndex - 1,
            choice
        );

        quickSort(
            movies,
            pivotIndex + 1,
            high,
            choice
        );
    }
}

int main() {

    const int MAX_MOVIES = 1000000;

    Movie* movies = new Movie[MAX_MOVIES];

    ifstream file("movies_sample.csv");

    if (!file) {
        cout << "Error: Could not open movies_sample.csv" << endl;
        delete[] movies;
        return 1;
    }

    string line;

    // Skip CSV header
    getline(file, line);

    int count = 0;

    // Read CSV file
    while (getline(file, line) && count < MAX_MOVIES) {

        stringstream ss(line);
        string value;

        // movie_id
        getline(ss, value, ',');
        movies[count].movieId = stoi(value);

        // title
        getline(ss, value, ',');
        movies[count].title = value;

        // genre
        getline(ss, value, ',');
        movies[count].genre = value;
```

```
        // release_year
        getline(ss, value, ',');
        movies[count].releaseYear = stoi(value);

        // imdb_rating
        getline(ss, value, ',');
        movies[count].imdbRating = stod(value);

        // watch_time_minutes
        getline(ss, value, ',');
        movies[count].watchTime = stoll(value);

        count++;
    }

    file.close();

    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "------------------------------------------------------------\n";
    cout << "Total movies read: " << count << endl;

    // Menu
    int choice;

    cout << "\nSort movies by:\n";
    cout << "1. IMDb Rating\n";
    cout << "2. Release Year\n";
    cout << "3. Watch Time Popularity\n";
    cout << "Enter your choice: ";
    cin >> choice;

    if (choice < 1 || choice > 3) {
        cout << "Invalid choice!" << endl;
        delete[] movies;
        return 1;
    }

    // Apply Quick Sort
    quickSort(movies, 0, count - 1, choice);

    cout << "\nMovies sorted successfully!\n";
    cout << "Showing first 20 movies:\n\n";

    // Display first 20 movies
    int displayCount = (count < 20) ? count : 20;

    for (int i = 0; i < displayCount; i++) {

        cout << "Movie ID: " << movies[i].movieId
             << " | Title: " << movies[i].title
             << " | Genre: " << movies[i].genre
             << " | Release Year: " << movies[i].releaseYear
             << " | IMDb Rating: " << movies[i].imdbRating
             << " | Watch Time: " << movies[i].watchTime
             << " minutes"
             << endl;
    }

    delete[] movies;

    return 0;
}
```

9. Sample Execution Output

```
=========================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 2 - Movie Recommendation Data Sorting using Quick Sort
=========================================================

Total movies read: 10000

Sort movies by:
1. IMDb Rating
2. Release Year
3. Watch Time Popularity
Enter your choice: 1

Movies sorted successfully!
Showing first 20 movies:

Movie ID: 7717 | Title: Endless Harbor 7717 | Genre: Documentary | Release Year: 1972 | IMDb Rating: 2.2
| Watch Time: 729705 minutes
Movie ID: 8285 | Title: Quiet Echo 8285 | Genre: Comedy | Release Year: 1971 | IMDb Rating: 2.3 | Watch
Time: 305343 minutes
Movie ID: 6282 | Title: Last Mirage 6282 | Genre: Drama | Release Year: 2000 | IMDb Rating: 2.3 | Watch
Time: 839727 minutes
Movie ID: 7229 | Title: Wild Shadow 7229 | Genre: Fantasy | Release Year: 2021 | IMDb Rating: 2.4 |
Watch Time: 330356 minutes
Movie ID: 2709 | Title: Neon Odyssey 2709 | Genre: Fantasy | Release Year: 2019 | IMDb Rating: 2.7 |
Watch Time: 322718 minutes
Movie ID: 3209 | Title: Quiet Echo 3209 | Genre: Fantasy | Release Year: 2010 | IMDb Rating: 2.7 | Watch
Time: 359870 minutes
Movie ID: 7790 | Title: Dark Descent 7790 | Genre: Thriller | Release Year: 2020 | IMDb Rating: 2.8 |
Watch Time: 392763 minutes
Movie ID: 1479 | Title: Wild Verdict 1479 | Genre: Fantasy | Release Year: 1997 | IMDb Rating: 2.9 |
Watch Time: 115837 minutes
Movie ID: 8576 | Title: Fallen Horizon 8576 | Genre: Drama | Release Year: 1991 | IMDb Rating: 3 | Watch
Time: 859587 minutes
Movie ID: 4528 | Title: Fallen Odyssey 4528 | Genre: Action | Release Year: 1977 | IMDb Rating: 3.1 |
Watch Time: 264729 minutes
Movie ID: 909 | Title: Eternal Horizon 909 | Genre: SciFi | Release Year: 2002 | IMDb Rating: 3.2 |
Watch Time: 676122 minutes
Movie ID: 1314 | Title: Fallen Journey 1314 | Genre: Animation | Release Year: 2004 | IMDb Rating: 3.2 |
Watch Time: 428728 minutes
Movie ID: 64 | Title: Eternal Journey 64 | Genre: Horror | Release Year: 2026 | IMDb Rating: 3.2 | Watch
Time: 572696 minutes
Movie ID: 1962 | Title: Wild Empire 1962 | Genre: Horror | Release Year: 2004 | IMDb Rating: 3.2 | Watch
Time: 823467 minutes
Movie ID: 2365 | Title: Hidden Echo 2365 | Genre: Thriller | Release Year: 1990 | IMDb Rating: 3.2 |
Watch Time: 461112 minutes
Movie ID: 8143 | Title: Quiet Storm 8143 | Genre: SciFi | Release Year: 2012 | IMDb Rating: 3.2 | Watch
Time: 196364 minutes
Movie ID: 7097 | Title: Forgotten Journey 7097 | Genre: Romance | Release Year: 1996 | IMDb Rating: 3.3
| Watch Time: 920817 minutes
Movie ID: 8633 | Title: Last Frontier 8633 | Genre: Romance | Release Year: 1996 | IMDb Rating: 3.3 |
Watch Time: 43761 minutes
Movie ID: 1042 | Title: Golden Shadow 1042 | Genre: Comedy | Release Year: 1973 | IMDb Rating: 3.3 |
Watch Time: 255274 minutes
Movie ID: 41 | Title: Endless Reflection 41 | Genre: Romance | Release Year: 1983 | IMDb Rating: 3.3 |
Watch Time: 594764 minutes
```

10. Conclusion

Quick Sort provides a highly memory-efficient, cache-friendly in-place sorting mechanism for recommendation streaming systems. With an average-case runtime of **O(n log n)** and negligible memory footprint (O(log n) stack frames), it enables the StreamFlix engine to dynamically rank catalog titles across IMDb ratings, release years, and watch durations without latency overhead.

GitHub Repository Link

> **Repository Link:** https://github.com/siddhesh160606/DAA_lab